Overleaf Viewer Technical Documentation

1. Introduction

Overleaf Viewer is a utility designed to bridge the gap between ephemeral Overleaf share links and stable, programmatic access to compiled PDF documents. Because Overleaf share links often require manual interaction or suffer from expiration/session requirements, this project provides a middleman service that programmatically triggers Overleaf's internal compile workflow.

By extracting the final PDF URL from an Overleaf public project, the viewer ensures that users receive a permanent, stable URL that consistently serves the latest compiled version of the document. This architecture reduces manual overhead and provides a reliable integration point for automated documentation pipelines.

2. System Architecture

The application is built using a serverless-compatible Next.js architecture leveraging the App Router. The design separates concerns into two distinct layers:

- **Client-Side UI:** A React-based interface responsible for accepting user input (Overleaf share URLs) and generating unique viewer tokens.
- **Server-Side Orchestration:** API routes that manage the lifecycle of an Overleaf project interaction. This includes handling CSRF tokens, session authentication, triggering remote compilation, and managing the resulting PDF lifecycle.

Data Flow: 1. User provides a Share URL to the **Frontend UI**. 2. The UI generates a unique token and links to the **PDF Viewer Page**. 3. The **API Route Handler** interceptor processes the token, performing scraping/API calls to Overleaf. 4. **NodeCache** stores session identifiers and final PDF URLs to reduce redundant load on Overleaf servers.

3. Getting Started

Prerequisites

- Node.js 18.x or higher
- npm or yarn
- An active Vercel account (for deployment)

Local Setup

1. Clone the repository: `git clone [repository-url]`
2. Install dependencies: `npm install`
3. Configure environment variables (if required for API headers).
4. Run the development server: `npm run dev`

Deployment

The project is optimized for the Vercel platform. Push your code to a linked GitHub repository, and Vercel will automatically trigger the build process for the Next.js App Router structure. Ensure `Vercel Analytics` is enabled in your dashboard for usage tracking.

4. Core Modules & Components

API Route Handler (`app/api/overleaf/[token]/route.js`)

This is the heart of the application. It orchestrates the sequence of operations required to interact with Overleaf: * **CSRF Handling:** Extracting and passing mandatory security tokens. * **Authentication/Access:** Negotiating access to the public project. * **Compilation Workflow:** Triggering the remote build process. * **Extraction:** Parsing the response to obtain the direct PDF stream/URL.

NodeCache Service (`lib/utils.js`)

An in-memory caching layer that mitigates the risk of rate-limiting by Overleaf. It stores: * Active CSRF tokens and cookies. * Project-specific identifiers. * The resolved final PDF URL (with a TTL to ensure "latest" versioning).

PDF Viewer Page (`app/view/[token]/page.js`)

A lightweight wrapper that embeds the resolved PDF URL within an `iframe`. It acts as the permanent endpoint for users.

5. API Reference

`GET /api/overleaf/[token]`

This endpoint retrieves the compiled PDF link for a given project.

- **Parameters:** `token` (The unique identifier generated during project submission).
- **Request Sequence:**
- Check `NodeCache` for existing valid PDF link.
- If missing, initiate the Overleaf handshake (Cookie + CSRF).
- Execute `lib/process.js` logic to trigger compilation.
- Parse the project state until the PDF resource is available.
- **Response:** A redirect or embedded stream of the compiled PDF.

6. Development Guide

Handling Web Structure Changes

Overleaf is a dynamic platform. If the internal API structure changes (e.g., HTML class names for the "Compile" button or header structures), update the scraping logic located in `lib/process.js`.

Extending the Cache

The `NodeCache` implementation should be monitored for memory usage. If the deployment scales significantly, consider moving the cache to an external store like Redis, maintaining the current `lib/utils.js` interface to minimize code changes.

Best Practices

- Always utilize `Axios` interceptors for consistent header management across scraping requests.

- Ensure all error states from Overleaf (e.g., compile errors) are caught and surfaced to the UI rather than failing silently.

7. Deployment & Maintenance

Troubleshooting

- **Compilation Errors:** If the PDF fails to load, check the logs for "Compile Timeout" or "Access Denied." These often stem from changes in Overleaf's authentication headers.
- **Rate Limiting:** If experiencing frequent 429 errors, increase the TTL settings in `NodeCache`.

Monitoring

Use the **Vercel Analytics** dashboard to monitor traffic patterns and ensure that API route executions are within the expected duration limits for serverless functions.